CSA 1737 – ARTIFICIAL INTELLIGENCE

Laboratory Record – Experiments 11 to 15

Ex. No. 11: 8-Queens Problem

AIM

To write a Python program to solve the 8-Queens problem using backtracking.

ALGORITHM

Step 1: Start the program.

Step 2: Represent the board as a list where board[row] stores the column of the queen in that row.

Step 3: Start with row 0 and try placing a queen in each column.

Step 4: A position is safe if no earlier queen is in the same column or on the same diagonal.

Step 5: If a position is safe, place the queen and move to the next row recursively.

Step 6: If no column is safe in a row, backtrack to the previous row and try the next column.

Step 7: When all 8 queens are placed, store the solution.

Step 8: Display the number of solutions and the first solution as a board.

Step 9: Stop the program.

CODE

```
# Experiment 11: 8-Queens problem using Backtracking
N = 8
solutions = []

def is_safe(board, row, col):
    for r in range(row):
        c = board[r]
        # same column or same diagonal
        if c == col or abs(c - col) == abs(r - row):
            return False
    return True

def place_queens(board, row):
    if row == N:
        solutions.append(board[:])
        return
    for col in range(N):
        if is_safe(board, row, col):
            board[row] = col
            place_queens(board, row + 1)
            board[row] = -1          # backtrack

place_queens([-1] * N, 0)

print("Total number of solutions for 8-Queens :", len(solutions))
print("\nFirst solution (Q = Queen):\n")
first = solutions[0]
for r in range(N):
```

```
    print(" ".join("Q" if first[r] == c else "." for c in range(N)))
print("\nQueen column positions (row by row):", first)
```

OUTPUT

```
Total number of solutions for 8-Queens : 92

First solution (Q = Queen):

Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .

Queen column positions (row by row): [0, 4, 7, 5, 2, 6, 1, 3]
```

RESULT

Thus the Python program to solve the 8-Queens problem was written and executed successfully; 92 solutions were found and the output was verified.

Ex. No. 12: Water Jug Problem

AIM

To write a Python program to solve the Water Jug problem using Breadth First Search.

ALGORITHM

Step 1: Start the program.

Step 2: Read the capacities of Jug X and Jug Y and the target amount.

Step 3: Start with both jugs empty (0, 0) and insert this state into a queue.

Step 4: Remove a state from the queue. If either jug contains the target amount, display the path and stop.

Step 5: Generate the next states: fill X, fill Y, empty X, empty Y, pour X into Y, pour Y into X.

Step 6: Insert every new (unvisited) state into the queue with its path.

Step 7: Repeat steps 4 to 6 until the target is reached or the queue becomes empty.

Step 8: Stop the program.

CODE

```
# Experiment 12: Water Jug Problem using BFS
from collections import deque

def water_jug(cap_x, cap_y, target):
    start = (0, 0)
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        (x, y), path = queue.popleft()
        if x == target or y == target:
            return path
        moves = [
            (cap_x, y),                                     # fill jug X
            (x, cap_y),                                     # fill jug Y
            (0, y),                                         # empty jug X
            (x, 0),                                         # empty jug Y
            (x - min(x, cap_y - y), y + min(x, cap_y - y)),   # pour X -> Y
            (x + min(y, cap_x - x), y - min(y, cap_x - x)),   # pour Y -> X
        ]
        for state in moves:
            if state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))
    return None

cap_x = int(input("Enter capacity of Jug X: "))
cap_y = int(input("Enter capacity of Jug Y: "))
target = int(input("Enter the target amount: "))

solution = water_jug(cap_x, cap_y, target)
if solution:
    print("\nSteps (Jug X, Jug Y):")
    for i, s in enumerate(solution):
        print("Step", i, ":", s)
```

```
else:
    print("No solution possible")
```

OUTPUT

```
Enter capacity of Jug X: 4
Enter capacity of Jug Y: 3
Enter the target amount: 2

Steps (Jug X, Jug Y):
Step 0 : (0, 0)
Step 1 : (0, 3)
Step 2 : (3, 0)
Step 3 : (3, 3)
Step 4 : (4, 2)
```

RESULT

Thus the Python program for the Water Jug problem was written and executed successfully, and the target of 2 litres was measured in 4 steps.

Ex. No. 13: Crypt-Arithmetic Problem

AIM

To write a Python program to solve the crypt-arithmetic problem SEND + MORE = MONEY.

ALGORITHM

Step 1: Start the program.

Step 2: Take the distinct letters of the puzzle: S, E, N, D, M, O, R, Y.

Step 3: Generate every possible assignment of distinct digits (0-9) to these letters using permutations.

Step 4: Reject assignments in which the leading letters S or M are zero.

Step 5: Convert the words SEND, MORE and MONEY into numbers using the assignment.

Step 6: If SEND + MORE = MONEY, display the assignment and the addition, then stop.

Step 7: Otherwise try the next assignment.

CODE

```
# Experiment 13: Crypt-Arithmetic Problem  (SEND + MORE = MONEY)
from itertools import permutations

letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')

def value(word, mapping):
    num = 0
    for ch in word:
        num = num * 10 + mapping[ch]
    return num

for perm in permutations(range(10), len(letters)):
    mapping = dict(zip(letters, perm))
    if mapping['S'] == 0 or mapping['M'] == 0:     # no leading zeros
        continue
    send = value("SEND", mapping)
    more = value("MORE", mapping)
    money = value("MONEY", mapping)
    if send + more == money:
        print("Solution found:")
        print(mapping)
        print("\n  ", send)
        print("+ ", more)
        print("-------")
        print(" ", money)
        break
```

OUTPUT

```
Solution found:
{'S': 9, 'E': 5, 'N': 6, 'D': 7, 'M': 1, 'O': 0, 'R': 8, 'Y': 2}

   9567
+  1085
-------
  10652
```

RESULT

Thus the Python program for the crypt-arithmetic problem was written and executed successfully: 9567 + 1085 = 10652.

Ex. No. 14: Missionaries and Cannibals Problem

AIM

To write a Python program to solve the Missionaries and Cannibals problem using Breadth First Search.

ALGORITHM

Step 1: Start the program.

Step 2: Represent a state as (missionaries on left bank, cannibals on left bank, boat side). The start state is (3, 3, left) and the goal state is (0, 0, right).

Step 3: The boat carries 1 or 2 people: (1M), (2M), (1C), (2C) or (1M, 1C).

Step 4: A state is valid only if on each bank the cannibals do not outnumber the missionaries (when missionaries are present).

Step 5: Insert the start state in a queue and repeatedly remove a state, generate all valid next states and insert the unvisited ones.

Step 6: When the goal state is reached, display the path of states.

Step 7: Stop the program.

CODE

```
# Experiment 14: Missionaries and Cannibals Problem using BFS
from collections import deque

# State = (missionaries on left, cannibals on left, boat side)  boat: 0 = left, 1
= right
def is_valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False
    if m > 0 and c > m:                  # left bank: cannibals outnumber
missionaries
        return False
    if (3 - m) > 0 and (3 - c) > (3 - m):  # right bank
        return False
    return True

def solve():
    start, goal = (3, 3, 0), (0, 0, 1)
    moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        (m, c, b), path = queue.popleft()
        if (m, c, b) == goal:
            return path
        for dm, dc in moves:
            if b == 0:
                nm, nc = m - dm, c - dc
            else:
                nm, nc = m + dm, c + dc
            state = (nm, nc, 1 - b)
            if is_valid(nm, nc) and state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))
```

```
path = solve()
print("State = (Missionaries left, Cannibals left, Boat side)\n")
for i, (m, c, b) in enumerate(path):
    side = "Left " if b == 0 else "Right"
    print(f"Step {i:2d}: Left = {m}M {c}C | Right = {3-m}M {3-c}C | Boat on
{side}")
print("\nTotal crossings:", len(path) - 1)
```

OUTPUT

```
State = (Missionaries left, Cannibals left, Boat side)

Step  0: Left = 3M 3C | Right = 0M 0C | Boat on Left
Step  1: Left = 3M 1C | Right = 0M 2C | Boat on Right
Step  2: Left = 3M 2C | Right = 0M 1C | Boat on Left
Step  3: Left = 3M 0C | Right = 0M 3C | Boat on Right
Step  4: Left = 3M 1C | Right = 0M 2C | Boat on Left
Step  5: Left = 1M 1C | Right = 2M 2C | Boat on Right
Step  6: Left = 2M 2C | Right = 1M 1C | Boat on Left
Step  7: Left = 0M 2C | Right = 3M 1C | Boat on Right
Step  8: Left = 0M 3C | Right = 3M 0C | Boat on Left
Step  9: Left = 0M 1C | Right = 3M 2C | Boat on Right
Step 10: Left = 1M 1C | Right = 2M 2C | Boat on Left
Step 11: Left = 0M 0C | Right = 3M 3C | Boat on Right

Total crossings: 11
```

RESULT

Thus the Python program for the Missionaries and Cannibals problem was written and executed successfully; all six people crossed the river in 11 crossings.

Ex. No. 15: Vacuum Cleaner Problem

AIM

To write a Python program to implement the Vacuum Cleaner problem for a two-room world using a simple reflex agent.

ALGORITHM

Step 1: Start the program.

Step 2: Create two rooms A and B, both Dirty, and place the vacuum agent in room A.

Step 3: If the current room is Dirty, perform the SUCK action and mark the room Clean.

Step 4: If the current room is Clean, MOVE to the other room.

Step 5: Display the state after each action.

Step 6: Repeat steps 3 to 5 until both rooms are Clean.

Step 7: Display the total number of actions performed.

Step 8: Stop the program.

CODE

```
# Experiment 15: Vacuum Cleaner Problem (two-room world, simple reflex agent)
# Room status: 'Dirty' or 'Clean'
rooms = {'A': 'Dirty', 'B': 'Dirty'}
location = 'A'
cost = 0

print("Initial state :", rooms, "| Agent at", location)
print()

while rooms['A'] == 'Dirty' or rooms['B'] == 'Dirty':
    if rooms[location] == 'Dirty':
        print(f"Room {location} is Dirty -> Action: SUCK")
        rooms[location] = 'Clean'
    else:
        new_loc = 'B' if location == 'A' else 'A'
        print(f"Room {location} is Clean -> Action: MOVE to {new_loc}")
        location = new_loc
    cost += 1
    print("   State :", rooms, "| Agent at", location)

print("\nBoth rooms are clean. Total actions performed =", cost)
```

OUTPUT

```
Initial state : {'A': 'Dirty', 'B': 'Dirty'} | Agent at A

Room A is Dirty -> Action: SUCK
   State : {'A': 'Clean', 'B': 'Dirty'} | Agent at A
Room A is Clean -> Action: MOVE to B
   State : {'A': 'Clean', 'B': 'Dirty'} | Agent at B
Room B is Dirty -> Action: SUCK
   State : {'A': 'Clean', 'B': 'Clean'} | Agent at B

Both rooms are clean. Total actions performed = 3
```

RESULT

Thus the Python program for the Vacuum Cleaner problem was written and executed successfully, and both rooms were cleaned in 3 actions.